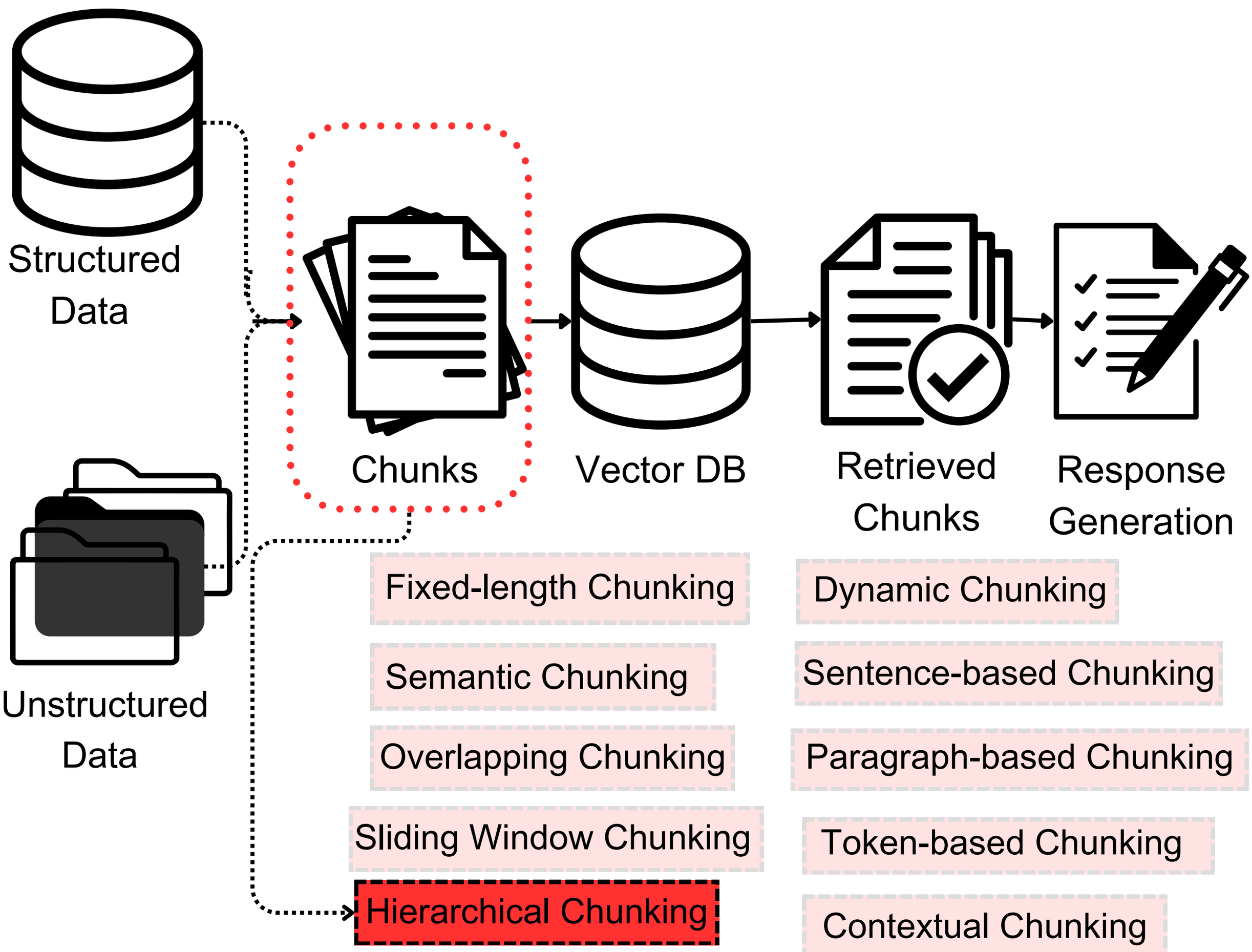
Different Types of Chunking in RAG System



Hierarchical

Chunking

Game-changer you need

What is Chunking?🤔

- Chunking is the process of breaking text into smaller, meaningful units.
- It ensures better manageability and improved context for NLP tasks.
- However, traditional chunking struggles with complex, multi-level data structures.

Hierarchical

Chunking

What is Hierarchical Chunking?

- Hierarchical Chunking takes text segmentation to the next level by organizing chunks into a multi-level structure.
- Instead of treating all text chunks as equal, this method creates nested layers of information, mimicking the document's logical flow.

For example

Level 1 → Document split into sections.

Level 2 → Sections split into paragraphs.

Level 3 → Paragraphs split into sentences.

Hierarchical Chunking

Three Key Aspects

- Preserve Context Across Levels: Maintains logical relationships between sections, paragraphs, and sentences.
- Enhanced Scalability: Efficiently handles large and complex datasets with multi-level hierarchies.
- Improved Semantic Understanding: Models capture the interplay between broader themes and granular details.

Where should you use?

1. **Multi-Level Summarization**: Generate concise summaries at different levels (e.g., document vs. paragraph).
2. **Question Answering**: Answer questions based on a specific section while retaining context from the whole document.
3. **Knowledge Graph Building**: Extract structured insights from complex documents with a clear hierarchy.

Hierarchical Chunking in Python

How it works?

1. **Level Sizes**: Define chunk sizes for each hierarchy level.
2. **Chunking Logic**: The text is chunked iteratively, with each level refining the granularity.
3. **Nested Structure**: A dictionary stores chunks at each level for easy hierarchical access.

```python
def hierarchical_chunking(text, level_sizes):
    """
    Segments the input text into a hierarchical structure of chunks.

    Args:
        text (str): The input text to chunk.
        level_sizes (list[int]): List of chunk sizes for each level in the hierarchy.

    Returns:
        dict: Hierarchical structure of text chunks.
    """
    def chunk_text(text, chunk_size):
        """Helper function to split text into chunks of a given size."""
        return [text[i:i + chunk_size] for i in range(0, len(text), chunk_size)]

    hierarchy = {}
    current_level_text = text

    for level, chunk_size in enumerate(level_sizes, start=1):
        chunks = chunk_text(current_level_text, chunk_size)
        hierarchy[f"Level {level}"] = chunks

        # For the next level, treat each chunk as a new text to be further chunked
        current_level_text = " ".join(chunks)  # Combine chunks for hierarchical processing

    return hierarchy
```

Hierarchical Chunking in Python

Output

```python
# Example Document
sample_document = (
    "Natural Language Processing (NLP) involves understanding and generating human language. "
    "It has applications in sentiment analysis, machine translation, and more. "
    "One challenge in NLP is segmenting long text streams into manageable pieces. "
    "Hierarchical chunking organizes text into multi-level structures for better understanding. "
    "This approach is useful for tasks like summarization, knowledge graph building, and question answering."
)

# Define hierarchical levels with respective chunk sizes (in characters)
level_sizes = [120, 60, 30]

# Perform hierarchical chunking
hierarchical_chunks = hierarchical_chunking(sample_document, level_sizes)

# Display the hierarchical chunks
for level, chunks in hierarchical_chunks.items():
    print(f"\n{level}:")
    for i, chunk in enumerate(chunks, start=1):
        print(f"  Chunk {i}: {chunk}")
```

✓ 0.0s

```
Level 1:
  Chunk 1: Natural Language Processing (NLP) involves understanding and generating human language. It has applications in sentiment
  Chunk 2:  analysis, machine translation, and more. One challenge in NLP is segmenting long text streams into manageable pieces. H
  Chunk 3: ierarchical chunking organizes text into multi-level structures for better understanding. This approach is useful for ta
  Chunk 4: sks like summarization, knowledge graph building, and question answering.
```

Hierarchical Chunking - LangChain

How it works?

1. **RecursiveCharacterTextSplitter**:
   a. Allows flexible chunking with overlap.
   b. Overlap is defined as 25% of the chunk size to ensure continuity.
2. **Hierarchical Levels**: Chunk sizes decrease as we move deeper into the hierarchy, capturing finer details.
3. **Dictionary Structure**: Each level is stored in a dictionary for organized access.

```python
from langchain.text_splitter import RecursiveCharacterTextSplitter

def hierarchical_chunking_with_langchain(text, level_sizes):
    """
    Hierarchical chunking using LangChain's RecursiveCharacterTextSplitter.

    Args:
        text (str): The input text to chunk.
        level_sizes (list[int]): List of chunk sizes for each level in the hierarchy.

    Returns:
        dict: A dictionary with hierarchical chunks at each level.
    """
    hierarchy = {}
    current_text = [text]  # Start with the full text

    for level, chunk_size in enumerate(level_sizes, start=1):
        level_chunks = []
        text_splitter = RecursiveCharacterTextSplitter(
            chunk_size=chunk_size,
            chunk_overlap=chunk_size // 4  # Overlap is 25% of the chunk size
        )

        # Chunk each piece of text from the previous level
        for chunk in current_text:
            level_chunks.extend(text_splitter.split_text(chunk))

        hierarchy[f"Level {level}"] = level_chunks
        current_text = level_chunks  # Use the current level chunks for the next level

    return hierarchy
```

Hierarchical Chunking - LangChain

```python
# Example Document
sample_document = (
    "Natural Language Processing (NLP) involves understanding and generating human language. "
    "It has applications in sentiment analysis, machine translation, and more. "
    "One challenge in NLP is segmenting long text streams into manageable pieces. "
    "Hierarchical chunking organizes text into multi-level structures for better understanding. "
    "This approach is useful for tasks like summarization, knowledge graph building, and question answering."
)

# Define hierarchical levels with respective chunk sizes (in characters)
level_sizes = [150, 100, 50]

# Perform hierarchical chunking
hierarchical_chunks = hierarchical_chunking_with_langchain(sample_document, level_sizes)

# Display the hierarchical chunks
for level, chunks in hierarchical_chunks.items():
    print(f"\n{level}:")
    for i, chunk in enumerate(chunks, start=1):
        print(f"  Chunk {i}: {chunk}")
```

✓ 0.0s

```
Level 1:
  Chunk 1: Natural Language Processing (NLP) involves understanding and generating human language. It has applications in sentiment analysis, machine
  Chunk 2: in sentiment analysis, machine translation, and more. One challenge in NLP is segmenting long text streams into manageable pieces. Hierarchical
  Chunk 3: into manageable pieces. Hierarchical chunking organizes text into multi-level structures for better understanding. This approach is useful for tasks
  Chunk 4: This approach is useful for tasks like summarization, knowledge graph building, and question answering.

Level 2:
  Chunk 1: Natural Language Processing (NLP) involves understanding and generating human language. It has
  Chunk 2: human language. It has applications in sentiment analysis, machine
  Chunk 3: in sentiment analysis, machine translation, and more. One challenge in NLP is segmenting long text
  Chunk 4: is segmenting long text streams into manageable pieces. Hierarchical
```

Benefits

1. **Flexibility:** Customizable chunk sizes and overlap.
2. **Scalability:** Handles large documents efficiently by structuring them hierarchically.
3. **Semantic Understanding:** Retains context across levels for better NLP performance.

CONGRATULATIONS

You have reached the end, now

If you want to help your network

REPOST THIS

Sarveshwaran R

https://github.com/DataSphereX/Chunking-Strategies

DataSphereX/**Chunking-Strategies**

1 Contributor | 0 Issues | 0 Stars | 0 Forks

DataSphereX/Chunking-Strategies

Contribute to DataSphereX/Chunking-Strategies development by creating an account on GitHub.

GitHub